

Cuaderno de laboratorio

Git, GitHub y RStudio

Una guía práctica para guardar, compartir y defender tu trabajo en R

Guía de consulta y práctica

La idea en una frase

Git es la bitácora de tu proyecto: guarda fotos ordenadas de tus cambios. GitHub es el lugar donde esa bitácora puede vivir en línea para que tu equipo la consulte, revise y mejore.

Hoy vas a aprender a:

- instalar y reconocer Git desde RStudio;
- crear un repositorio en GitHub y traerlo a tu computadora;
- hacer commits pequeños y con mensajes claros;
- subir cambios con push y actualizar con pull;
- trabajar con ramas, pull requests y conflictos sencillos;
- usar un ejemplo mínimo de análisis de señales en R.

Dra. Antonieta Martínez Guerrero / 2026

Mapa rápido de la guía

	Cómo usar este cuaderno No intentes memorizar Git. Úsalo como una receta: lee el paso, ejecútalo, revisa el resultado y marca el checkpoint. Cuando algo falle, vuelve a git status: casi siempre te dirá qué está pasando.	
1	Kit de instalación	Las ligas oficiales y lo que debes tener listo antes de abrir RStudio.
2	Ideas mínimas	Git, GitHub, commit, rama, pull request y conflicto en lenguaje cotidiano.
3	Preparar tu computadora	Identidad de Git, conexión con GitHub y revisión desde RStudio.
4	Primer repositorio con RStudio	Crear, clonar, editar, guardar, subir y revisar.
5	Ramas y pull requests	Cómo proponer cambios sin romper el proyecto principal.
6	Conflictos sin drama	Qué significan las marcas de conflicto y cómo resolverlas.
7	Tres ejercicios	Una práctica sencilla y dos intermedias para terminar la sesión.
8	Chuleta final	Comandos, errores frecuentes y checklist de supervivencia.
	Tu ritual durante toda la clase • Antes de empezar a trabajar: git pull. • Después de editar: git status. • Cuando algo ya funciona: git add + git commit. • Cuando quieres compartirlo: git push.	

1. Kit de instalación

Meta de esta sección

Que tu computadora tenga tres piezas conversando entre sí: R para calcular, RStudio para trabajar cómodamente y Git para guardar historia. GitHub será la nube donde compartirás esa historia.

Descargas oficiales

Git: <https://git-scm.com/install/> - elige Windows, macOS o Linux según tu equipo

R desde CRAN: <https://cran.r-project.org/> - instala primero R; RStudio lo usa por debajo

RStudio Desktop: <https://posit.co/download/rstudio-desktop/> - usa la edición gratuita de escritorio

Cuenta de GitHub: <https://github.com/signup> - crea una cuenta con correo que puedas revisar

Ligas de consulta para cuando practiques en casa

Guía de control de versiones en RStudio: <https://docs.posit.co/ide/user/ide/guide/tools/version-control.html>

Happy Git and GitHub for the useR: <https://happygitwithr.com/> - excelente recurso para R + GitHub

Git Cheat Sheet oficial: <https://git-scm.com/cheat-sheet>

Crear un repositorio en GitHub: <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository>

Clonar un repositorio: <https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository>

Crear un pull request: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request>

Orden recomendado de instalación

Instala / crea	Por qué importa
R	Es el lenguaje que usaremos para los ejemplos.
RStudio Desktop	Es el espacio de trabajo donde editaremos scripts y veremos
Git	Es la herramienta que registra el historial del proyecto.
Cuenta de GitHub	Permite subir, compartir y revisar cambios en línea.

Checkpoint de instalación

Abre RStudio. En la parte inferior busca la pestaña Terminal. Ahí escribe `git --version`. Si aparece un número de versión, Git ya responde.

En la Terminal de RStudio

```
git --version
```

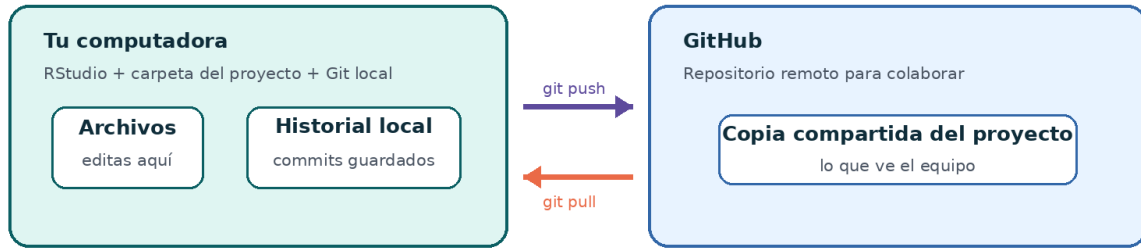
2. Ideas mínimas antes de teclear

Analogía de laboratorio

Un proyecto de análisis no avanza en línea recta: pruebas una gráfica, corriges una función, comparas resultados y a veces regresas. Git permite que ese proceso quede documentado sin depender de carpetas como

version_final_ahora_si_2.

Dónde viven tus cambios



No todo cambio llega a GitHub: primero debe estar en un commit local; luego se sube con push.

Git no es lo mismo que GitHub

Concepto	Qué es	Ejemplo cotidiano
Git	Programa instalado en tu computadora que registra cambios.	La bitácora del laboratorio.
GitHub	Plataforma web que aloja repositorios y facilita colaboración.	El estante compartido donde el equipo consulta la bitácora.
Repositorio	Carpeta de proyecto con historial Git.	La libreta completa de un experimento.
Commit	Foto guardada de un conjunto pequeño de cambios.	Una entrada fechada en la bitácora.
Rama	Línea alternativa de trabajo.	Una prueba que no quieres mezclar todavía.
Pull request	Solicitud para revisar y unir una rama.	Pedir revisión antes de aceptar cambios.
Conflicto	Dos personas cambiaron la misma parte de un archivo.	Dos notas distintas escritas sobre la misma línea.

Los cuatro lugares por los que pasa un cambio

Lugar	Qué ocurre ahí	Comando asociado
Carpeta de trabajo	Editas archivos en RStudio.	<code>git status</code> para ver qué cambió.
Zona de preparación	Eliges qué cambios irán en la próxima foto.	<code>git add</code> archivo o checkbox en RStudio.
Historial local	Guardas la foto con un mensaje.	<code>git commit -m "mensaje".</code>
GitHub remoto	Compartes tus commits con el equipo.	<code>git push</code> ; para traer cambios, <code>git pull</code> .

El ciclo cotidiano de Git



Idea clave: un commit es una foto clara; un push es compartir esa foto en GitHub.

Regla de oro

Haz commits pequeños. Un buen commit debe poder leerse como una decisión: "agrega función para simular ruido", "corrige título de gráfica", "documenta variables".

3. Preparar tu computadora

Objetivo

Que Git sepa quién eres, que RStudio encuentre Git y que GitHub pueda autenticarte de forma segura.

3.1 Configura tu identidad de Git

Ejecuta esto una sola vez en la Terminal de RStudio. Usa el nombre y correo que quieres que aparezcan en tus commits.

```
git config --global user.name "Tu Nombre Apellido"
git config --global user.email "tu_correo@ejemplo.com"
git config --global init.defaultBranch main
git config --list --global
```

No uses comillas raras

Copia comillas rectas como estas: " ". Algunas comillas tipográficas pueden causar errores en la Terminal.

3.2 Revisa que RStudio vea Git

1. En RStudio entra a Tools > Global Options > Version Control o Git/SVN.
2. Activa Enable version control interface if it is not already active.
3. Si Git executable aparece vacío, busca git.exe o git en tu instalación de Git.
4. Cierra y vuelve a abrir RStudio si acabas de instalar Git.

3.3 Prepara paquetes útiles de R

En la Consola de R

```
install.packages(c("usethis", "gitcreds", "here"))
usethis::git_sitrep()
```

Autenticación con GitHub

GitHub ya no acepta la contraseña normal de la cuenta para operaciones Git por HTTPS. Lo más cómodo suele ser iniciar sesión en el navegador cuando Git Credential Manager lo pida, o usar un Personal Access Token guardado de forma segura. Nunca pegues tokens en un script ni los subas a GitHub.

Si necesitas guardar un token con R

En la Consola de R

```
# Abre instrucciones para crear un token:
usethis::create_github_token()

# Guarda el token en el administrador de credenciales del sistema:
gitcreds::gitcreds_set()
```

OK	Checkpoint
[]	git --version muestra una versión.
[]	git config --list --global muestra tu nombre y correo.
[]	RStudio tiene activada la interfaz de control de versiones.
[]	Tienes cuenta de GitHub y puedes iniciar sesión en el navegador.

4. Primer repositorio con RStudio

Lo que haremos

Crearemos un repositorio en GitHub, lo clonaremos en RStudio, escribiremos un script mínimo de R, guardaremos un commit y lo subiremos a GitHub.

4.1 Crea el repositorio en GitHub

1. Entra a GitHub y pulsa New repository.
2. Nombre sugerido: git-rstudio-senales.
3. Marca Add a README file.
4. En .gitignore elige R si aparece la opción; si no, lo agregaremos después.
5. Crea el repositorio y copia la URL HTTPS.

Nombre del repositorio

Usa minúsculas y guiones: git-rstudio-senales. Evita espacios, acentos y nombres como tarea final final.

4.2 Clona el repositorio en RStudio

1. File > New Project > Version Control > Git.
2. Pega la URL HTTPS del repositorio.
3. Elige una carpeta fácil de encontrar.
4. Pulsa Create Project. RStudio abrirá un proyecto nuevo.

Equivalente en Terminal

```
git clone https://github.com/TU_USUARIO/git-rstudio-senales.git
cd git-rstudio-senales
git status
```

4.3 Crea una estructura mínima

En la Consola de R

```
dir.create("scripts", showWarnings = FALSE)
dir.create("datos", showWarnings = FALSE)
dir.create("figuras", showWarnings = FALSE)
```

4.4 Agrega tu primer script de R

Crea el archivo scripts/01_senal_basica.R y pega este código. Es deliberadamente sencillo: una señal senoidal y una gráfica guardada como imagen.

```
# scripts/01_senal_basica.R
# Primera señal: una senoidal limpia

dir.create("figuras", showWarnings = FALSE)

tiempo <- seq(0, 1, length.out = 200)
frecuencia <- 5
senal <- sin(2 * pi * frecuencia * tiempo)

plot(
  tiempo, senal,
  type = "l",
  main = "Señal senoidal de 5 Hz",
  xlab = "Tiempo (s)",
  ylab = "Amplitud"
)

png("figuras/senal_basica.png", width = 900, height = 500)
plot(
  tiempo, senal,
  type = "l",
  main = "Señal senoidal de 5 Hz",
  xlab = "Tiempo (s)",
  ylab = "Amplitud"
)
dev.off()
```

4.5 Documenta el README

En README.md

```
# git-rstudio-senales

Repositorio de práctica para aprender Git, GitHub y RStudio.

## Primer ejemplo
Simulamos una señal senoidal de 5 Hz y guardamos una figura en la carpeta `figuras/`.

## Integrantes
- Tu nombre
```

4.6 Guarda la primera foto del proyecto

En la Terminal de RStudio

```
git status
git add README.md scripts/01_senal_basica.R figuras/senal_basica.png
git commit -m "Agrega primera señal senoidal en R"
git push
```

Lo mismo desde la pestaña Git de RStudio

- Marca los archivos que quieres guardar en el commit.
- Pulsa Commit.

	• Escribe un mensaje breve y claro. • Pulsa Commit y luego Push.
OK	Checkpoint
☐	El repositorio existe en GitHub.
☐	El proyecto abre en RStudio como proyecto .Rproj.
☐	El script 01_senal_basica.R corre sin errores.
☐	GitHub muestra README.md, scripts/01_senal_basica.R y figuras/senal_basica.png.

5. Ramas y pull requests

	Idea central Una rama es una zona de prueba. Trabajas ahí, confirmas que el cambio sirve y después pides que se revise mediante un pull request. Así main se mantiene como la versión estable.
--	--

Flujo con ramas

```
git pull
git switch -c mejora-ruido
# editar archivos en RStudio
git status
git add scripts/02_senal_con_ruido.R README.md
git commit -m "Agrega simulación de señal con ruido"
git push -u origin mejora-ruido
```

Código para la rama mejora-ruido


```
# scripts/02_senal_con_ruido.R
# Señal limpia + ruido gaussiano

dir.create("datos", showWarnings = FALSE)
dir.create("figuras", showWarnings = FALSE)

set.seed(123)
tiempo <- seq(0, 1, length.out = 300)
frecuencia <- 5
senal_limpia <- sin(2 * pi * frecuencia * tiempo)
ruido <- rnorm(length(tiempo), mean = 0, sd = 0.25)
senal_ruidosa <- senal_limpia + ruido

datos <- data.frame(
  tiempo = tiempo,
  senal_limpia = senal_limpia,
  senal_ruidosa = senal_ruidosa
)

write.csv(datos, "datos/senal_ruidosa.csv", row.names = FALSE)

png("figuras/senal_con_ruido.png", width = 900, height = 500)
plot(tiempo, senal_ruidosa, type = "l",
     main = "Señal de 5 Hz con ruido",
     xlab = "Tiempo (s)", ylab = "Amplitud")
lines(tiempo, senal_limpia, lwd = 2)
legend("topright", legend = c("Ruidosa", "Limpia"), lwd = c(1, 2))
dev.off()

summary(datos)
```

Crear el pull request en GitHub

1. Entra al repositorio en GitHub.
2. Busca el aviso Compare & pull request o entra a la rama mejora-ruido.
3. Escribe qué cambiaste y por qué.
4. Pide revisión a una compañera o compañero.
5. Cuando se apruebe, pulsa Merge pull request.

Cómo escribir un buen pull request

- Qué agregué: una frase concreta.
- Cómo probarlo: qué script correr.
- Qué resultado espero: archivo, gráfica o tabla.
- Qué duda tengo: algo que quieras que revisen.

Después de que se acepte el pull request

```
git switch main
git pull
git branch
```

6. Conflictos sin drama

Qué es un conflicto

Git puede combinar muchos cambios automáticamente. Pero si dos personas modifican la misma línea de un mismo archivo, Git se detiene y te pide decidir qué versión conservar.

Cómo se ve un conflicto

Dentro del archivo en conflicto

```
<<<<<< HEAD
Objetivo: analizar una señal limpia de 5 Hz.
=====
Objetivo: analizar una señal con ruido y comparar resultados.
>>>>>> mejora-ruido
```

Marca	Significa
<<<<<< HEAD	Tu versión actual.
=====	Separador entre versiones.
>>>>>> nombre-rama	La otra versión que Git intentaba combinar.

Receta para resolver

1. Abre el archivo marcado con conflicto.
2. Lee ambas versiones.
3. Escribe una versión final clara; puede combinar ideas de ambas.
4. Borra todas las marcas <<<<<<, ===== y >>>>>>.
5. Guarda el archivo.
6. Ejecuta git status para confirmar qué falta.
7. Marca el conflicto como resuelto con git add.
8. Cierra con commit y push.

```
git status
git add README.md
git commit -m "Resuelve conflicto en README"
git push
```

No borres a ciegas

Resolver un conflicto no significa elegir siempre arriba o abajo. Significa escribir una versión final que tenga sentido para el proyecto.

7. Práctica de una hora: tres ejercicios

Forma de trabajo

Los tres ejercicios están diseñados para completarse en aproximadamente 60 minutos. Trabaja con calma: cada ejercicio deja una evidencia visible en GitHub.

Ejercicio 1 - Sencillo: mi primer commit reproducible (12 minutos)

Misión

Crear un repositorio individual con un script de R que genere una señal senoidal y subirlo a GitHub.

Acción

Crea un repositorio llamado practica-git-rstudio con README.

Evidencia

URL del repositorio.

Clónalo desde RStudio.

Proyecto abierto en RStudio.

Crea scripts/01_senal_basica.R con el código de la sección 4.

Script corre sin errores.

Actualiza README.md con objetivo e integrante.

README legible.

Haz add, commit y push.

GitHub muestra el commit.

Comandos sugeridos

```
git status
git add README.md scripts/01_senal_basica.R figuras/senal_basica.png
git commit -m "Agrega señal senoidal básica"
git push
```

OK	Checkpoint
[]	Mi repositorio tiene README.md.
[]	Mi script está dentro de scripts/.
[]	Mi commit tiene un mensaje claro.
[]	Mi repositorio en GitHub está actualizado.

Ejercicio 2 - Intermedio: trabajo en pareja con rama y pull request (25 minutos)**Misión**

Agregar un análisis de señal con ruido desde una rama, pedir revisión y unir el cambio con pull request.

Rol**Responsabilidad**

Persona A

Crea o usa el repositorio, invita a la persona B como colaboradora y revisa el pull request.

Persona B

Clona el repositorio, crea la rama, agrega el script de ruido, hace push y abre el pull request.

Pasos técnicos

1. Persona A: en GitHub, Settings > Collaborators, invita a Persona B.
2. Persona B: clona el repositorio y ejecuta git pull.
3. Persona B: crea una rama llamada analisis-ruido.
4. Persona B: agrega scripts/02_senal_con_ruido.R y actualiza README.md.
5. Persona B: commit y push de la rama.
6. Persona B: abre pull request en GitHub.
7. Persona A: revisa, comenta una mejora pequeña y acepta el pull request.
8. Ambas personas: vuelven a main y ejecutan git pull.

Comandos para Persona B

```
git pull
git switch -c analisis-ruido
# Crear o editar scripts/02_senal_con_ruido.R
git status
git add README.md scripts/02_senal_con_ruido.R datos/senal_ruidosa.csv figuras/senal_con_ruido.png
git commit -m "Agrega análisis de señal con ruido"
git push -u origin analisis-ruido
```

Evidencia del ejercicio 2

- URL del pull request.
- Al menos un comentario de revisión.
- Pull request aceptado o, si no alcanzó el tiempo, pull request abierto con cambios visibles.

Ejercicio 3 - Intermedio: conflicto controlado en README (23 minutos)

Misión

Provocar un conflicto pequeño en README.md, resolverlo y dejar constancia en GitHub. El objetivo es practicar sin miedo.

Preparación

Antes de empezar, ambas personas deben estar en main y con el repositorio actualizado:

```
git switch main
git pull
git status
```

Provocar el conflicto

1. Persona A: en GitHub, edita una línea del README, por ejemplo el objetivo del proyecto. Haz commit directo en main.
2. Persona B: sin hacer pull, edita esa misma línea del README en RStudio con una versión diferente.
3. Persona B: guarda, hace add y commit local.
4. Persona B: intenta traer los cambios de GitHub con git pull --no-rebase origin main.
5. Git debería mostrar conflicto en README.md.
6. Persona B: resuelve el conflicto escribiendo una versión final que combine ambas ideas.
7. Persona B: hace add, commit y push.

Comandos del conflicto

```
# Persona B, después de editar README.md localmente
git add README.md
git commit -m "Edita objetivo del README"
git pull --no-rebase origin main

# Resolver marcas de conflicto en README.md, guardar y luego:
git status
git add README.md
git commit -m "Resuelve conflicto del objetivo del proyecto"
git push
```

Evidencia del ejercicio 3

- README final sin marcas de conflicto.
- Commit con mensaje de resolución.
- Historial en GitHub mostrando los cambios.

Entrega sugerida de la práctica

Qué entregar	Dónde se ve
URL del repositorio individual	GitHub, ejercicio 1.
URL del pull request	GitHub, ejercicio 2.
Captura o URL del commit de conflicto resuelto	GitHub, ejercicio 3.

Qué entregar	Dónde se ve
Reflexión de 3 líneas	En README.md: qué comando te resultó más útil y por qué.

8. Chuleta final de comandos

		Usa esta página cuando te atoraste El primer comando casi siempre es git status. Te dice en qué rama estás, qué archivos cambiaron y qué espera Git de ti.
Comando	Cuándo usarlo	
git status	Ver qué cambió y qué falta por hacer.	
git add archivo	Preparar un archivo para el siguiente commit.	
git add .	Preparar todos los cambios de la carpeta actual. Úsalo con cuidado.	
git commit -m "mensaje"	Guardar una foto del proyecto con mensaje.	
git push	Subir commits a GitHub.	
git pull	Traer cambios de GitHub a tu computadora.	
git switch main	Cambiar a la rama main.	
git switch -c nombre-rama	Crear una rama nueva y cambiarte a ella.	
git branch	Listar ramas locales.	
git diff	Ver cambios antes de agregarlos.	
git diff --staged	Ver cambios ya preparados para commit.	
git log --oneline --graph --decorate --all	Ver historial compacto con ramas.	
git restore archivo	Descartar cambios no preparados de un archivo.	
git restore --staged archivo	Quitar un archivo de la zona de preparación sin borrar cambios.	

Mensajes de commit que sí ayudan

Poco útil	Mejor
cambios	Agrega simulación de señal senoidal
final	Documenta estructura del repositorio
arreglos varios	Corrige ruta de figura en README
prueba	Agrega gráfica de señal con ruido

Problemas frecuentes

Lo que ves	Qué significa	Qué hacer
git: command not found	Git no está instalado o RStudio no lo encuentra.	Instala Git, reinicia RStudio y revisa Tools > Global Options.

Lo que ves	Qué significa	Qué hacer
nothing to commit	No hay cambios nuevos o ya fueron guardados.	Revisa git status y confirma que editaste/guardaste el archivo.
Authentication failed	GitHub no aceptó credenciales.	Usa navegador/Git Credential Manager o PAT; no uses contraseña normal.
push rejected	GitHub tiene cambios que tú no tienes.	Ejecuta git pull, resuelve si hay conflicto y vuelve a push.
Aparecen <<<<<< en un archivo	Hay conflicto de combinación.	Edita una versión final, borra marcas, git add, commit, push.
No veo la pestaña Git en RStudio	El proyecto no está conectado a un repositorio Git.	Abre el proyecto clonado o revisa que exista la carpeta .git.

Checklist antes de pedir ayuda

- Ejecuté git status y leí el mensaje completo.
- Estoy en la rama correcta.
- Guardé los archivos en RStudio antes de hacer commit.
- Estoy dentro de la carpeta del repositorio.
- No subí contraseñas, tokens ni archivos innecesarios.

Anexo. Archivos que conviene ignorar

Un archivo .gitignore le dice a Git qué no debe rastrear. Para proyectos sencillos de R puedes usar algo como esto:

.gitignore

```
# Archivos de sesión de R
.Rhistory
.RData
.Rproj.user/

# Archivos temporales del sistema
.DS_Store
Thumbs.db

# Resultados que pueden regenerarse, si tu profesor lo indica
# figuras/
# datos/
```

Cuidado con datos sensibles

Antes de subir datos a GitHub, pregunta si se pueden compartir. GitHub público significa público. Si hay datos personales, clínicos, contraseñas, llaves o tokens: no los subas.

Mini bitácora personal

El comando que más voy a usar es:

Lo que todavía me confunde es:

La próxima vez que trabaje en equipo, antes de editar voy a:

Mi repositorio de práctica está en:

Fuentes y documentación oficial

Git - instalación: <https://git-scm.com/install/>

CRAN - descarga de R: <https://cran.r-project.org/>

Posit - RStudio Desktop: <https://posit.co/download/rstudio-desktop/>

RStudio User Guide - Version Control: <https://docs.posit.co/ide/user/ide/guide/tools/version-control.html>

Git Cheat Sheet oficial: <https://git-scm.com/cheat-sheet>

GitHub Docs - crear repositorio: <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository>

GitHub Docs - clonar repositorio: <https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository>

GitHub Docs - crear pull request: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request>

GitHub Docs - conflictos: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github>

GitHub Docs - autenticación: <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github>

usethis - credenciales Git/GitHub: <https://usethis.r-lib.org/articles/git-credentials.html>

Cierre

Git se aprende como una rutina. No necesitan memorizar todos los comandos desde el primer día. Lo importante es acostumbrarse a cuatro acciones básicas: hacer **pull** antes de empezar, usar **status** para saber qué está pasando, hacer **commit** cuando algo ya funciona y usar **push** para compartir el avance en GitHub.